T20 Optimisation des Tokens

Technique Intermediate

Réduire la consommation de tokens sans perdre en qualité

RTK : Filtrage en Amont

RTK (Rust Token Killer) intercepte les outputs de commandes **avant** qu'ils entrent dans le contexte de Claude. Ce n'est pas un résumé post-hoc, c'est un filtre qui élimine le bruit structurellement redondant.

```
# Installation
brew install rtk
# ou : cargo install --git https://github.com/rtk-ai/rtk

# Inspecter les changements de configuration avant l'intégration
rtk init --dry-run
rtk init
```

Mesurer la réduction des sorties

Comparer des commandes équivalentes sur le même état du dépôt, inspecter les échecs et utiliser `rtk gain` pour les estimations locales. Réduire la sortie du shell ne couvre qu'une partie de la consommation de tokens ; cela ne prouve pas une réduction identique de la facture totale. RTK estime les tokens sans exécuter le tokenizer du modèle. Voir le calcul des économies RTK.

Commandes RTK essentielles

```
rtk git status      # Git condensé
rtk git log         # Log sans bruit
rtk cargo test      # Tests Rust filtrés
rtk vitest run      # Tests JS condensés
rtk gh pr view 123  # PR GitHub optimisé
rtk gain            # Dashboard d'économies
```

Compact : Libérer le Contexte

`/compact` résume l'historique de conversation. L'espace récupéré varie et des détails peuvent être perdus ; conserver les décisions et prochaines étapes dans des fichiers et inspecter `/context` avant de continuer.

```
/compact            # Résumer la session en cours
```

La différence avec `/clear` : compact conserve la continuité de la session, clear repart de zéro.

Limiter les lectures et la recherche

Claude Code ne documente aucun mécanisme natif `.claudeignore`. Nommer les répertoires pertinents dans les demandes. Pour bloquer un accès, utiliser les règles de permission de `.claude/settings.json`, par exemple :

```
{
  "permissions": {
    "deny": ["Read(./node_modules/**)", "Read(./dist/**)"]
  }
}
```

Examiner séparément les accès shell et MCP. Ces règles restreignent les accès ; elles ne classent pas les résultats de recherche. Voir les patterns de permissions.

Règles de Prompt Efficaces

Cibler, pas charger :

- « Check the `login` function in `auth.ts:45-60` » plutôt que « Check auth.ts for issues »
- Référencer des symboles (`calculateTotal`) plutôt que des fichiers entiers
- Éviter de coller de gros JSON ou logs bruts dans le prompt

Ce qui consomme le plus :

Action	Ce qu'il faut mesurer
Lecture d'un gros fichier	Taille du texte retourné
Appel MCP	Schémas des outils et résultat renvoyé
Longue conversation	Historique conservé dans le contexte
Exécution des tests	Sortie retournée, échecs compris

Stratégie Combinée

Filtrer les sorties verbeuses du shell, compacter lorsque c'est utile et limiter les lectures aux sources pertinentes. Vérifier que le résultat conserve les preuves nécessaires pour terminer la tâche.